

Resumen técnico — CodelabsWeb

Table of Contents

1. Contexto y metadatos	1
2. Propósito funcional	2
3. Arquitectura	2
4. Interfaces y contratos	3
5. Flujos técnicos	4
6. Seguridad y contexto de ejecución	4
7. Persistencia	5
8. Catálogo de implementación	5
9. Operación y configuración	5
10. Riesgos y deuda	6
Anexo — Evidencia técnica	7
Dependencias declaradas	7
Punto de entrada de la aplicación	8
Composición de rutas y componentes	8
Internacionalización propia (sin librería de terceros)	9
Infraestructura AWS CDK (<i>infra/</i>)	9
Patrones notables	10
Anexo — Arquitectura y flujos técnicos	10
Flujo 1: resolución de locale y navegación bilingüe	10
Flujo 2: pipeline de CI/CD con despliegue condicional	11

1. Contexto y metadatos

Repositorio: [CodelabsWeb](#). Fuente: `origin/master` refrescado (worktree detached limpio). SHA: `51c55321ef036b5896a219709ab88539fdf76c04`.

La rama remota `origin/master`, después de `git fetch origin master`, es la fuente de verdad del código y de los cambios integrados. El estado y los diffs de Git identifican el trabajo local pendiente; los planes de continuidad y la memoria de las herramientas de IA son auxiliares.

La documentación formal es canónica en [Documentación/](#) y se genera desde un worktree detached limpio del master remoto. `provenance.json` registra el commit fuente: el commit que publica los documentos es posterior al commit documentado.

Tecnología: Next.js/React/TypeScript; versiones declaradas en `package.json`: `next ^15.5.9`, `react ^18.3.1`, `react-dom ^18.3.1`, `typescript ^5`, `tailwindcss ^4` (vía `@tailwindcss/postcss`), `eslint ^9` con `eslint-config-next 15.5.9`. Fuente de fuente tipográfica: `geist ^1.7.2`. Node fijado con Volta: `20.19.6`. La infraestructura AWS CDK vive en `infra/` como paquete independiente (`@codelabs/web-infra`, `aws-`

cdk-lib ^2.224.0, constructs ^10.4.3, aws-cdk ^2.1033.0).

Estructura versionada inmediata del repositorio (raíz del worktree): `app/`, `infra/`, `public/`, `social/`, `.github/`, `Documentación/`, `README.md`, `next.config.ts`, `package.json`, `package-lock.json`, `tsconfig.json`, `eslint.config.mjs`, `postcss.config.mjs`, `.gitignore`, `.gitattributes`.

Inventario de código fuente TypeScript/TSX (excluyendo `node_modules` y `infra/cdk.out`): 48 archivos — 44 bajo `app/`, 3 bajo `infra/` (`bin/codelabs-web.ts`, `lib/config.ts`, `lib/static-site-stack.ts`) y 1 en la raíz (`next.config.ts`). No se encontraron archivos de prueba (`.test.`, `.spec.`, `tests/`) en ninguna ruta del repositorio.

Contexto multi-IA, según la política versionada en `README.md`: el repositorio adopta el protocolo global 2.2; Codex carga `AGENTS.md` y Claude Code carga `CLAUDE.md`, cuyo único contenido es `@AGENTS.md`. Las instrucciones globales de ambas herramientas mantienen el mismo protocolo. `AGENTS.md`, `CLAUDE.md` y `docs/exec-plans/` son estado local excluido mediante el archivo privado `info/exclude`; no se versionan ni se añaden a `.gitignore`. El contexto local debe instalarse en cada checkout donde se trabaje.

2. Propósito funcional

CodelabsWeb es el sitio corporativo estático de CodeLabs Ecuador (empresa de ingeniería de software), construido con Next.js App Router y exportado como HTML/CSS/JS estático (`output: "export"` en `next.config.ts:4`). No expone backend propio ni rutas de API: toda interacción de contacto delega en enlaces `mailto:` y `wa.me` (WhatsApp) resueltos en el cliente.

Rutas reales verificadas en `app/` (ver matriz completa en `matrices/interfaces.csv`):

- Sitio en español (grupo de rutas (`site`)): `/` (`app/(site)/page.tsx`), `/servicios` (`app/(site)/servicios/page.tsx`), `/casos` (`app/(site)/casos/page.tsx`), `/nosotros` (`app/(site)/nosotros/page.tsx`), `/contacto` (`app/(site)/contacto/page.tsx`).
- Sitio en inglés, bajo el mismo grupo (`site`): `/en` (`app/(site)/en/page.tsx`), `/en/services`, `/en/case-studies`, `/en/about`, `/en/contact`.
- Páginas legales (grupo de rutas (`legal`), en español únicamente): `/privacidad`, `/tratamiento-datos`, `/terminos`, `/desuscripcion`.

El mapeo español↔inglés de rutas equivalentes está codificado explícitamente en `app/i18n/paths.ts:3-9` (`pathPairs`), no mediante enrutamiento de Next.js basado en segmentos [`locale`].

Alcance verificado: presentación institucional (propuesta de valor, capacidades, casos de éxito, stack tecnológico, clientes), captación de contacto (formulario que abre WhatsApp), y contenido legal (privacidad, tratamiento de datos, términos, desuscripción) conforme a LOPDP (Ecuador). No hay autenticación, panel de administración, ni persistencia de datos de negocio.

3. Arquitectura

Next.js App Router con dos grupos de rutas paralelos que comparten el layout raíz:

- `app/layout.tsx` (raíz): define metadata global (título, OpenGraph, Twitter card, íconos, manifest — líneas 6-48), fuente `GeistSans`, monta `<html lang="es">` fijo y el componente cliente `HtmlLangSync` (`app/ui/html-lang-sync.tsx`) que sincroniza `document.documentElement.lang` en cada navegación según la ruta (`app/ui/html-lang-sync.tsx:7-15`).
- `app/(site)/layout.tsx`: layout del sitio público — envuelve el contenido con `Navbar`, `Footer` y el botón flotante `WhatsAppButton` (líneas 1-18).
- `app/(legal)/*`: páginas legales autocontenidas, sin layout adicional propio (usan `<main>` directamente, p. ej. `app/(legal)/privacidad/page.tsx`).

Composición por secciones: cada página compone componentes de presentación reutilizables desde `app/(site)/sections/` (17 archivos: hero, trust-bar, value-prop, clients, capabilities/-teaser, case-studies/-teaser, cta-banner, contact/-info, navbar, footer, about, aws-showcase, how-we-work, tech-stack). Los datos de contenido (clientes, capacidades, casos, stack) están en módulos TypeScript planos bajo `app/data/`, no en un CMS ni API externa.

Internacionalización: implementación propia por diccionario (`app/i18n/dictionary.ts`, 580 líneas) indexado por `Locale` (`"es" | "en"`, `app/i18n/locale.ts`), sin librería de i18n de terceros (no usa `next-intl` ni `next-i18next`). Cada componente cliente que necesita texto localizado recibe la prop `locale` y llama `getDictionary(locale)`.

Estilos: Tailwind CSS 4 vía `@import "tailwindcss"` en `app/globals.css:1` y el plugin `@tailwindcss/postcss` (`postcss.config.mjs`); variables de marca (`--primary`, `--surface`, etc.) definidas en `:root` (`app/globals.css:3-11`).

Infraestructura (`infra/`, paquete AWS CDK v2 independiente del build de Next.js): `infra/bin/codelabs-web.ts` instancia una única stack `StaticSiteStack` (`infra/lib/static-site-stack.ts`) usando configuración de `infra/lib/config.ts`, que solo admite el stage `"prod"` (lanza excepción para cualquier otro valor — `infra/lib/config.ts:36-42`). La stack compone: bucket S3 privado, distribución CloudFront con Origin Access Control, certificado ACM en `us-east-1`, registros alias Route 53, una CloudFront Function de reescritura de URLs limpias, y un rol IAM asumible vía OIDC de GitHub Actions. Detalle completo en `modules/evidencia.adoc`.

Publicación: pipeline único en `.github/workflows/pipeline.yml` que construye el export estático (`npm run build → out/`) y lo sincroniza a S3 + invalidación de CloudFront cuando corresponde. Flujo completo en `modules/arquitectura_y_flujos.adoc`.

4. Interfaces y contratos

Sitio estático sin rutas de API de servidor: no existen archivos `route.ts/route.tsx` (Route Handlers) ni directorio `pages/api` en el repositorio. Las únicas "interfaces" son las páginas HTML que Next.js exporta estáticamente y dos integraciones de salida hacia servicios externos, ambas resueltas en el cliente sin backend propio:

- `mailto:info@codelabsecuador.com` (enlaces directos en varias páginas y secciones).
- `https://wa.me/593{teléfono}` (botón flotante `app/ui/whatsapp-button.tsx:13` y envío del formulario de contacto `app/(site)/sections/contact.tsx:14-28`, que arma el mensaje con los campos capturados y lo abre en una pestaña nueva vía `window.open`).

Inventario completo de páginas/rutas con archivo y línea en `matrices/interfaces.csv`.

5. Flujos técnicos

Dos flujos representativos, documentados con cita de archivo y línea en `modules/arquitectura_y_flujos.adoc`:

1. **Resolución de locale y navegación bilingüe:** `app/i18n/paths.ts` mapea rutas ES↔EN (`pathPairs`, líneas 3-9) y expone `localeFromPathname` (línea 11-12) y `getAlternatePath` (líneas 15-19); `HtmlLangSync` (`app/ui/html-lang-sync.tsx`) consume `localeFromPathname` en cada cambio de ruta para mantener el atributo `lang` del documento sincronizado, ya que el layout raíz fija `<html lang="es">` de forma estática (`app/layout.tsx:56`).
2. **Pipeline de CI/CD con despliegue condicional:** `.github/workflows/pipeline.yml` — job `changes` detecta rutas modificadas y decide qué validar/desplegar (líneas 64-184); `web-quality` hace lint + build + valida artefactos del export (líneas 186-221); `infra-validate` hace `tsc --noEmit` + `cdk synth` (líneas 223-251); `deploy-web` solo corre si `deploy_web=true` (push a `master` o `workflow_dispatch`) y ambos gates previos pasaron o se omitieron, sube `_next/static` con cache inmutable y el resto con cache corta, e invalida CloudFront (líneas 253-306).

6. Seguridad y contexto de ejecución

No hay autenticación ni autorización en la aplicación: es un sitio público sin sesiones, roles ni datos protegidos.

CloudFront (`infra/lib/static-site-stack.ts`): TLS mínimo `SecurityPolicyProtocol.TLS_V1_2_2021` (línea 134), `ViewerProtocolPolicy.REDIRECT_TO_HTTPS` (línea 115), origen S3 servido únicamente vía Origin Access Control (`S3BucketOrigin.withOriginAccessControl`, línea 113 — el bucket no es accesible públicamente), y `ResponseHeadersPolicy.SECURITY_HEADERS` (línea 114), la política administrada de AWS para cabeceras de seguridad estándar. El código de este repositorio no define una política de cabeceras personalizada ni una Content-Security-Policy explícita; solo referencia la política administrada por nombre.

S3 (`infra/lib/static-site-stack.ts:72-79`): `BlockPublicAccess.BLOCK_ALL`, `BucketEncryption.S3_MANAGED`, `enforceSSL: true`, `ObjectOwnership.BUCKET_OWNER_ENFORCED`, `versioned: true`, `RemovalPolicy.RETAIN` (el bucket no se destruye si se elimina la stack).

Despliegue desde GitHub Actions: sin credenciales estáticas. El rol `GitHubActionsDeployRole` (`infra/lib/static-site-stack.ts:174-231`) se asume vía OIDC (`token.actions.githubusercontent.com`) con condición `StringEquals` que restringe el sub a `repo:FrancisU20/CodelabsWeb:ref:refs/heads/master` y `repo:FrancisU20/CodelabsWeb:environment:production` (líneas 190-193), con permisos IAM acotados a `s3:PutObject/DeleteObject/GetBucketLocation/ListBucket/PutObjectTagging` sobre el bucket del sitio y `cloudfront:CreateInvalidation` sobre la distribución concreta (líneas 210-226). El job `deploy-web` del pipeline usa el `environment: production` de GitHub (línea 262) y permisos mínimos `id-token: write` (línea 265).

No se detectaron secretos, claves ni credenciales en el código fuente del worktree. La configuración

sensible de despliegue (rol a asumir, nombre del bucket, ID de distribución) se referencia por nombre de variable de repositorio de GitHub (`AWS_ROLE_TO_ASSUME`, `SITE_BUCKET_NAME`, `CLOUDFRONT_DISTRIBUTION_ID`, `AWS_REGION`), nunca por valor, tal como documenta [infra/README.md](#).

7. Persistencia

No aplica: sitio estático sin persistencia propia. El bucket S3 de [infra/](#) almacena únicamente los artefactos estáticos del export (`out/`) como hosting, no datos de negocio ni estado de aplicación. El formulario de contacto (`app/(site)/sections/contact.tsx`) no envía datos a ningún backend propio: construye un mensaje y abre un enlace [wa.me](#) en el cliente; no hay almacenamiento, cola ni base de datos involucrados.

8. Catálogo de implementación

Verificado por conteo directo sobre el worktree:

- Páginas (`page.tsx`): 14 — 5 en español + 5 en inglés (grupo `(site)`) + 4 legales (grupo `(legal)`).
- Componentes de sección (`app/(site)/sections/*.tsx`): 17.
- Componentes de UI compartidos (`app/ui/*.tsx`): 4 — `html-lang-sync.tsx`, `logo.tsx`, `page-intro.tsx`, `whatsapp-button.tsx`.
- Módulos de datos estáticos (`app/data/*.ts`): 4 — `capabilities.ts`, `case-studies.ts`, `clients.ts`, `stack.ts`.
- Módulos de i18n (`app/i18n/*.ts`): 3 — `locale.ts`, `paths.ts`, `dictionary.ts`.
- Layouts: `app/layout.tsx` (raíz), `app/(site)/layout.tsx`.
- Infraestructura CDK: 1 entry point (`infra/bin/codelabs-web.ts`), 1 construct de stack (`infra/lib/static-site-stack.ts`), 1 módulo de configuración (`infra/lib/config.ts`).
- No se encontraron suites de prueba (unitarias, de integración o end-to-end) en el repositorio.

Detalle narrativo por componente/módulo en [modules/evidencia.adoc](#); inventario tabular en [matrices/components.csv](#).

9. Operación y configuración

Comandos declarados en `package.json` (raíz, app web): `npm ci` (setup), `npm run dev` (next dev), `npm run build` (next build, genera export estático en `out/` por `output: "export"`), `npm run start` (serve out), `npm run lint` (eslint).

Comandos declarados en `infra/package.json`: `npm run build` (`tsc --noEmit`), `npm run synth` (`cdk synth`), `npm run diff` (`cdk diff`), y `npm run cdk` como passthrough a la CLI de CDK. `infra/cdk.json` fija el entry point (`npx ts-node --prefer-ts-exts bin/codelabs-web.ts`) y activa el contexto `@aws-cdk/aws-cloudfront:defaultSecurityPolicyTLSv1.2_2021`.

Pipeline real de publicación (`.github/workflows/pipeline.yml`, con las acciones compuestas `.github/actions/setup-web/action.yml` y `.github/actions/setup-infra/action.yml`): se dispara en pull

request contra `master`, en push a `master` y por `workflow_dispatch` manual. Detecta cambios por ruta para decidir si valida web, valida infra, o despliega; el despliegue real a S3/CloudFront solo ocurre en push a `master` o dispatch manual con `deploy_web=true`, y requiere el `environment: production` de GitHub configurado con las variables de repositorio `AWS_ROLE_TO_ASSUME`, `AWS_REGION`, `SITE_BUCKET_NAME`, `CLOUDFRONT_DISTRIBUTION_ID` (usadas en `pipeline.yml:57-61` y validadas explícitamente antes de desplegar en la línea 273-278).

Variables de entorno/configuración leídas por el código (solo nombres, sin valores): `CDK_DEFAULT_ACCOUNT`, `CDK_DEFAULT_REGION`/región fija `us-east-1` (`infra/lib/config.ts:20-21`), `GITHUB_OIDC_PROVIDER_ARN` (override opcional, `infra/lib/config.ts:29-33`). El perfil de AWS local esperado para operaciones manuales de CDK es `code-labs` (documentado en `README.md` e `infra/README.md`, no forzado por código).

La generación documental parte del commit remoto identificado en `provenance.json` y solo actualiza fuentes y derivados de `Documentación/`. Los controles de publicación de la aplicación siguen registrados en el pipeline; generar documentación no acredita por sí mismo una ejecución de esos controles ni un despliegue.

10. Riesgos y deuda

Clasificación	Hallazgo	Archivo	Línea
Deuda técnica	Datos de contacto (teléfono <code>0984522092</code> y correo <code>info@code-labsecuador.com</code>) duplicados como literales en al menos 9 ubicaciones distintas (componentes, diccionario <code>i18n</code> y páginas legales) sin una única fuente de verdad	<code>app/ui/whatsapp-button.tsx</code> ; <code>app/(site)/sections/contact.tsx</code> ; <code>app/(site)/sections/contact-info.tsx</code> ; <code>app/i18n/dictionary.ts</code> ; <code>app/(legal)/privacidad/page.tsx</code> ; <code>app/(legal)/terminos/page.tsx</code>	<code>whatsapp-button.tsx:12</code> ; <code>contact.tsx:14</code> ; <code>contact-info.tsx:79</code> ; <code>dictionary.ts:143,310,368,535</code> ; <code>privacidad/page.tsx:191</code> ; <code>terminos/page.tsx:37,205</code>
Riesgo / cobertura	No existen pruebas automatizadas (unitarias, de integración o end-to-end) en el repositorio; la validación de calidad en CI se limita a lint y a que el build de Next.js no falle	Repositorio completo (búsqueda <code>.test.</code> , <code>.spec.</code> , <code>tests/</code> sin resultados)	No aplica (ausencia verificada)

Clasificación	Hallazgo	Archivo	Línea
Limitación de diseño	La infraestructura CDK solo admite el stage "prod"; cualquier otro valor de <code>stage</code> lanza una excepción, por lo que no existe un ambiente de staging para validar cambios de infraestructura antes de producción	<code>infra/lib/config.ts</code>	<code>config.ts:36-42</code>
Observación de seguridad	La distribución CloudFront usa la política administrada <code>ResponseHeadersPolicy.SECURITY_HEADERS</code> de AWS; el repositorio no define una Content-Security-Policy propia ni una política de cabeceras personalizada	<code>infra/lib/static-site-stack.ts</code>	<code>static-site-stack.ts:114</code>

Anexo — Evidencia técnica

Fuente de toda la evidencia de este anexo: `worktree detached` de `origin/master` en `51c55321ef036b5896a219709ab88539fdf76c04`.

Dependencias declaradas

Aplicación web (`package.json`, raíz):

- Runtime: `next ^15.5.9`, `react ^18.3.1`, `react-dom ^18.3.1`, `geist ^1.7.2`.
- Desarrollo: `typescript ^5`, `tailwindcss ^4`, `@tailwindcss/postcss ^4`, `eslint ^9`, `eslint-config-next 15.5.9`, `@types/node ^20`, `@types/react ^18.3.27`, `@types/react-dom ^18.3.7`, `serve ^14.2.6`.
- Node fijado vía Volta: `20.19.6` (`package.json`, bloque `volta`).

Infraestructura (`infra/package.json`, paquete `@code-labs/web-infra` independiente):

- Runtime: `aws-cdk-lib ^2.224.0`, `constructs ^10.4.3`.
- Desarrollo: `aws-cdk ^2.1033.0`, `ts-node ^10.9.2`, `typescript ^5.9.3`, `@types/node ^20.19.25`.

Ambos `package.json` son manifiestos separados (`tsconfig.json` de la app excluye explícitamente `infra` en su bloque `exclude`), por lo que la app web y la infraestructura CDK tienen árboles de dependencias y compilación independientes dentro del mismo repositorio.

Punto de entrada de la aplicación

`app/layout.tsx` es el layout raíz de Next.js App Router: define metadata SEO/OpenGraph/Twitter (líneas 6-48), registra la fuente `GeistSans` vía `geist/font/sans` (línea 2), fija `<html lang="es">` de forma estática (línea 56) y monta el componente cliente `HtmllangSync` antes de renderizar `children` (línea 62).

`next.config.ts` configura el modo de export estático:

```
const nextConfig: NextConfig = {
  output: "export",
  images: { unoptimized: true },
  eslint: { ignoreDuringBuilds: true },
};
```

(`next.config.ts:3-10`). `output: "export"` implica que no hay servidor Next.js en producción — el resultado de `next build` es HTML/CSS/JS estático en `out/`, servido íntegramente por CloudFront/S3. `images.unoptimized: true` es consistente con esa restricción (el optimizador de imágenes de Next.js requiere un servidor). `eslint.ignoreDuringBuilds: true` desacopla el lint del build de Next.js; el pipeline de CI ejecuta `npm run lint` como paso separado y previo al build (`.github/workflows/pipeline.yml:200-204`), por lo que el lint sigue siendo un gate real, solo que no bloquea `next build` en ejecuciones locales.

Composición de rutas y componentes

Grupos de rutas de Next.js (paréntesis = no afectan la URL):

- `app/(site)/` — sitio público, con layout propio (`app/(site)/layout.tsx`) que monta `Navbar`, `main` y `Footer` + `WhatsAppButton` flotante.
 - `app/(site)/page.tsx`, `servicios/`, `casos/`, `nosotros/`, `contacto/` — español.
 - `app/(site)/en/page.tsx`, `en/services/`, `en/case-studies/`, `en/about/`, `en/contact/` — inglés, bajo el mismo grupo de rutas y layout.
- `app/(legal)/` — `privacidad/`, `tratamiento-datos/`, `terminos/`, `desuscripcion/`, sin layout adicional propio.

Secciones reutilizables en `app/(site)/sections/` (17 archivos): `hero.tsx`, `trust-bar.tsx`, `value-prop.tsx`, `clients.tsx`, `capabilities.tsx`, `capabilities-teaser.tsx`, `case-studies.tsx`, `case-studies-teaser.tsx`, `cta-banner.tsx`, `contact.tsx`, `contact-info.tsx`, `navbar.tsx`, `footer.tsx`, `about.tsx`, `aws-showcase.tsx`, `how-we-work.tsx`, `tech-stack.tsx`. Cada página de `(site)` importa un subconjunto de estas secciones y les pasa la prop `locale` ("es" o "en" según el archivo de página).

Componentes compartidos en `app/ui/`: `logo.tsx` (wordmark SVG con variante clara/oscura, `next/image`), `page-intro.tsx` (encabezado reutilizado por páginas internas), `html-lang-sync.tsx` (sincroniza `<html lang>`), `whatsapp-button.tsx` (botón flotante global).

Datos de contenido estático en `app/data/`: `capabilities.ts`, `case-studies.ts` (estructura

`Record<"es"|"en", string>` por campo — ver ejemplo abajo), `clients.ts`, `stack.ts`. No hay llamadas a APIs externas ni CMS: todo el contenido textual vive en estos módulos TypeScript y en `app/i18n/dictionary.ts`.

```
export type CaseStudy = {
  id: string;
  title: Record<"es" | "en", string>;
  result: Record<"es" | "en", string>;
  impact: Record<"es" | "en", string>;
  focus: Record<"es" | "en", string>;
};
```

(`app/data/case-studies.ts:1-7`).

Internacionalización propia (sin librería de terceros)

`app/i18n/locale.ts` define el tipo `Locale = "es" | "en"` y el locale por defecto `"es"` (líneas 1-5). `app/i18n/paths.ts` mapea explícitamente cada ruta española a su equivalente en inglés en un arreglo `pathPairs` (líneas 3-9) y expone dos funciones puras:

- `localeFromPathname(pathname)`: determina el locale por prefijo `/en` (líneas 11-12).
- `getAlternatePath(pathname, target)`: resuelve la ruta equivalente en el locale destino para el selector de idioma (líneas 15-19), usada por `navbar.tsx` para el switch de idioma y por `HtmlLangSync`.

`app/i18n/dictionary.ts` (580 líneas) exporta un tipo `Dictionary` con secciones por área de UI (`nav`, `footer`, `hero`, `trustBar`, `valueProp`, `contact`, `whatsapp`, etc.) y una función `getDictionary(locale)` que devuelve el diccionario completo para ese locale. Los componentes cliente (p. ej. `contact.tsx`, `whatsapp-button.tsx`) llaman `getDictionary(locale)` para obtener sus textos.

Infraestructura AWS CDK (`infra/`)

Entry point (`infra/bin/codelabs-web.ts:1-13`): crea la `cdk.App`, lee el contexto `stage` (por defecto `"prod"`) e instancia `StaticSiteStack` con la configuración resuelta por `getEnvironmentConfig(stage)`.

`infra/lib/config.ts` define `StaticSiteConfig` y una única configuración real, `prodConfig` (líneas 16-34): `stackName: "CodelabsWebProdStack"`, región `us-east-1`, dominio `codelabsecuador.com` + alterno `www.codelabsecuador.com`, y los datos del repositorio de GitHub (`githubOwner: "FrancisU20"`, `githubRepository: "CodelabsWeb"`, `githubBranch: "master"`) usados para restringir el rol OIDC. `getEnvironmentConfig` lanza `Error` para cualquier `stage` distinto de `"prod"` (líneas 36-42): no hay ambiente de staging modelado en código.

`infra/lib/static-site-stack.ts` (`StaticSiteStack`, líneas 56-241) compone, en este orden:

1. `HostedZone.fromLookup` sobre `hostedZoneDomainName` (línea 62).
2. `Certificate ACM` con validación DNS sobre esa zona, incluyendo el dominio alterno (líneas 66-70).

3. **Bucket** S3 privado: `BLOCK_ALL`, cifrado gestionado por S3, SSL forzado, `BUCKET_OWNER_ENFORCED`, versionado, `RemovalPolicy.RETAIN` (líneas 72-79).
4. **Function** de CloudFront (CloudFront Functions, no Lambda@Edge) que reescribe URLs limpias a `.html` o `index.html` para soportar el export estático de Next.js (líneas 81-99).
5. **Distribution** con esa función asociada al evento `VIEWER_REQUEST`, origen S3 vía Origin Access Control, `ResponseHeadersPolicy.SECURITY_HEADERS`, `CachePolicy.CACHING_OPTIMIZED`, redirección forzada a HTTPS, TLS mínimo 1.2 (2021), HTTP/2 y HTTP/3, páginas de error 403/404 mapeadas a `/404.html` (líneas 101-136).
6. Registros `ARecord/AaaaRecord` alias hacia la distribución para cada dominio configurado (líneas 138-153).
7. Rol IAM `GitHubActionsDeployRole` asumible vía OIDC de GitHub Actions, con permisos acotados a subir/borrar objetos en el bucket del sitio e invalidar la distribución CloudFront concreta (método `createGitHubDeployRole`, líneas 174-231).

Salidas (`CfnOutput`, líneas 157-171): URL del sitio, nombre del bucket, ID y dominio de la distribución CloudFront, y ARN del rol de despliegue — estos valores son los que se cargan manualmente como variables de repositorio de GitHub según [infra/README.md](#).

Patrones notables

- Separación estricta build-time/deploy-time: el export estático (`out/`) se construye una sola vez en el job `web-quality` y se reutiliza como artefacto (`actions/upload-artifact / download-artifact`) en el job `deploy-web`, en lugar de reconstruirlo — evita divergencia entre lo validado y lo publicado (`.github/workflows/pipeline.yml:214-221` y `267-271`).
- Cache-Control diferenciado en el despliegue: los assets versionados de Next.js bajo `_next/static` se suben con `public,max-age=31536000,immutable`, mientras el resto del sitio usa `public,max-age=300,must-revalidate` (`.github/workflows/pipeline.yml:287-300`), coherente con el hashing de archivos que aplica `next build`.
- Sin llaves de acceso de AWS estáticas en ningún flujo: tanto el pipeline de GitHub Actions como la propia stack CDK están diseñados alrededor de un proveedor OIDC (`infra/lib/static-site-stack.ts:179-193`, `.github/workflows/pipeline.yml:280-285`).

Anexo — Arquitectura y flujos técnicos

Flujo 1: resolución de locale y navegación bilingüe

El sitio no usa el mecanismo de rutas dinámicas `[locale]` de Next.js ni middleware de negociación de idioma; el bilingüismo es un mapeo manual entre dos árboles de páginas (español en la raíz de `(site)`, inglés bajo `(site)/en/`).

1. `app/i18n/paths.ts:3-9` declara el mapa canónico de rutas equivalentes:

```
const pathPairs: Array<[es: string, en: string]> = [
  ["/", "/en"],
```

```
[
  "/servicios", "/en/services",
  "/casos", "/en/case-studies",
  "/nosotros", "/en/about",
  "/contacto", "/en/contact",
];
```

2. `localeFromPathname(pathname)` (`app/i18n/paths.ts:11-12`) clasifica cualquier ruta como "en" si es `/en` o empieza con `/en/`, y "es" en cualquier otro caso. Es la única fuente de verdad para "en qué idioma estoy" fuera de la página misma.
3. Cada `page.tsx` invoca sus secciones pasando el `locale` correspondiente como literal (p. ej. `app/(site)/page.tsx:19-25` pasa `locale="es"` a `Hero`, `TrustBar`, `ValueProp`, etc.; su contraparte `app/(site)/en/page.tsx` pasa `locale="en"`). El locale no se infiere en tiempo de ejecución dentro de la página: está fijado por en qué archivo vive.
4. Los componentes cliente que sí necesitan inferir el locale en tiempo de ejecución (porque no reciben la prop, p. ej. el botón flotante global) usan `usePathname()` + `localeFromPathname` directamente: `app/ui/whatsapp-button.tsx:9-11` y `app/ui/html-lang-sync.tsx:8-11`.
5. `HtmlLangSync` (montado una sola vez en `app/layout.tsx:62`, fuera de los grupos de rutas) corrige en el cliente el atributo `lang` del `<html>`, que el layout raíz fija de forma estática en "es" (`app/layout.tsx:56`):

```
useEffect(() => {
  document.documentElement.lang = localeFromPathname(pathname);
}, [pathname]);
```

(`app/ui/html-lang-sync.tsx:10-12`). Esto significa que en el HTML servido estáticamente (antes de hidratación) el atributo `lang` siempre es "es", incluso para `/en/*`; se corrige después, en el cliente, tras el primer render.

6. El selector de idioma de `navbar.tsx` usa `getAlternatePath(pathname, target)` (`app/i18n/paths.ts:15-19`) para calcular a qué URL enlazar al cambiar de idioma, buscando la ruta actual en `pathPairs` y devolviendo su par.

Flujo 2: pipeline de CI/CD con despliegue condicional

Pipeline único (`.github/workflows/pipeline.yml`) disparado por `pull_request` contra `master`, `push` a `master` y `workflow_dispatch` manual (líneas 20-41).

1. **changes (Plan / Detect Changes)**, líneas 64-184: hace checkout completo (`fetch-depth: 0`), calcula el rango `BASE..HEAD` según el tipo de evento (líneas 86-99) y clasifica cada archivo modificado en `web`, `infra`, `pipeline`, `docs` u `other` según su ruta (líneas 109-128). A partir de esas banderas deriva cuatro salidas booleanas consumidas por los jobs siguientes: `validate_web`, `validate_infra`, `deploy_web`, `docs_only` (líneas 130-154). Un cambio en `.github/workflows/` o `.github/actions/` (`pipeline=true`) fuerza la validación de ambas capas, web e infra, aunque el cambio en sí no toque ni `app/` ni `infra/`.
2. **web-quality (Web / Quality Gate)**, líneas 186-221, condicionado a `validate_web == 'true'`:

instala dependencias con la acción compuesta `setup-web` (`npm ci`), corre `npm run lint`, luego `npm run build` (genera `out/`), y valida que existan las páginas legales exportadas (`out/index.html`, `out/terminos.html`, `out/privacidad.html`, `out/tratamiento-datos.html`, `out/desuscripcion.html` — líneas 206-212). Solo si además `deploy_web == 'true'` sube `out/` como artefacto de build (`web-export-<sha>`, líneas 214-221).

3. **infra-validate (Infra / Validate CDK)**, líneas 223-251, condicionado a `validate_infra == 'true'`: instala dependencias de `infra/` con `setup-infra`, corre `tsc --noEmit` (vía `npm run build` dentro de `infra/`) y `cdk synth -c stage=prod --quiet`, y sube el/los template(s) CloudFormation resultantes como artefacto. Este job **valida** la infraestructura pero nunca ejecuta `cdk deploy` — el despliegue de infraestructura es manual (confirmado también por `infra/README.md`, que documenta `cdk deploy -c stage=prod --profile codelabs` como comando local).
4. **deploy-web (Deploy / Static Site)**, líneas 253-306: solo corre si `deploy_web == 'true'` y el gate `web-quality` fue exitoso (con `infra-validate` exitoso u omitido — condición `always()` con chequeo explícito de resultados, líneas 257-261). Usa el `environment: production` de GitHub y permisos `id-token: write` para autenticarse en AWS vía OIDC (`aws-actions/configure-aws-credentials`, líneas 280-285) sin credenciales estáticas. Sube primero los assets inmutables de `_next/static` con `--cache-control "public,max-age=31536000,immutable"` (líneas 287-293), luego el resto del sitio con `--cache-control "public,max-age=300,must-revalidate"` excluyendo `_next/static/` para no pisar la política de cache anterior (líneas 295-300), y finalmente invalida toda la distribución CloudFront (`--paths "/"`, líneas 302-306).
5. **summary (Pipeline / Summary)**, líneas 308-326: siempre corre (`if: always()`) y escribe un resumen legible de banderas y resultados en `GITHUB_STEP_SUMMARY`, independientemente de si hubo despliegue.

El detector solicita rutas literales a Git mediante `git -c core.quotePath=false diff --name-only` (línea 101), lo que permite reconocer el nombre acentuado `Documentación/`. Clasifica como documentación `AGENTS.md`, `CLAUDE.md`, `README.md`, `docs/exec-plans/`, `Documentación/` y `*.md` (línea 121). Las categorías previas mantienen prioridad: por ejemplo, `infra/README.md` se clasifica como infraestructura.

En push y pull request, un cambio puramente documental ejecuta `changes` y `summary` sin activar los gates de la aplicación ni su despliegue. Una ejecución manual con `deploy_web=true` solicita explícitamente build y publicación incluso cuando el rango de cambios es documental (líneas 143-145). Los cambios mixtos activan los controles de las categorías afectadas.